Demonstration: from what the API gave us to a dataset

Rohan Alexander

23 October, 2026

Introduction

Last week the Open Data Toronto API gave us the 500 most recent rows of the daily shelter occupancy data. Every one of its 32 columns arrived as a string. This week we turn that into a dataset, following the workflow in Chapter 9 of Alexander (2023). We save the original, plan and simulate, start small, write tests, and then document what we made so that someone else could use it (Gebru et al. 2021; Lewis 2024).

The order matters, and it is not the order that feels natural. We write the tests, then we simulate data that pass them, and only then do we look at the real data.

```
uv sync
uv run scripts/00-simulate.py
uv run scripts/01-clean.py
```

```
library(arrow)
library(dplyr)
library(ggplot2)
library(tinytable)
```

Plan

A row is a program on a night. We want to know when, which program, who it serves, how many people were there, how much room there was, and how much of it was used.

The raw data complicate that last part. Some programs count capacity in beds and some in rooms, and the source handles this with two parallel sets of columns, one of which is always empty.

```
raw <- read_parquet("data/raw/shelter_latest_500.parquet")

raw |>
  count(CAPACITY_TYPE,
        beds_missing = is.na(CAPACITY_ACTUAL_BED),
        rooms_missing = is.na(CAPACITY_ACTUAL_ROOM))
```

```
# A tibble: 2 x 4
  CAPACITY_TYPE       beds_missing rooms_missing     n
  <chr>               <lgl>        <lgl>         <int>
1 Bed Based Capacity  FALSE        TRUE            372
2 Room Based Capacity TRUE         FALSE           128
```

That is the "one thing per column" rule from Lewis (2024), and the first of the messy forms in Wickham (2014), where column headers are values (here, bed and room), in a real dataset. It is a deliberate choice. The City's description of the dataset says that room-based capacity is for family and hotel programs, where a room is not shared between households, and that reporting it separately prevents capacity from being over-reported. So it is a sensible way to publish the data. But it is awkward to analyze. Our plan is one column that says what the unit is, and then one column for each of capacity, occupied, and the occupancy rate.

Tests, written first

The tests are in their own file, `scripts/tests.py`, so that exactly the same tests run against the simulated data and the real data. They begin with the schema, which is the plan written as code.

```
SECTORS = ["Families", "Men", "Mixed Adult", "Women", "Youth"]
CAPACITY_UNITS = ["bed", "room"]

SCHEMA = {
    "occupancy_date": pl.Date,
    "program_id": pl.String,  # an identifier, not a number
    "sector": pl.String,
    "capacity_unit": pl.String,
    "service_users": pl.Int64,
    "capacity": pl.Int64,
    "occupied": pl.Int64,
    "occupancy_rate": pl.Float64,
}
```

Then, for every variable, a test of class and a test of content.

```
def test_shelter(df: pl.DataFrame) -> None:
    # Class: every variable is there, and is the type that we planned
    assert dict(df.schema) == SCHEMA, dict(df.schema)

    # Content: every variable takes values that it could take
    assert df["program_id"].str.contains(r"^\d{5}$").all()
    assert df["sector"].is_in(SECTORS).all()
    assert df["capacity_unit"].is_in(CAPACITY_UNITS).all()
    assert (df["service_users"] >= 0).all()
    assert (df["capacity"] > 0).all()
    assert df["occupancy_rate"].is_between(0, 100).all()
```

And then the tests that are worth the most, because they say something that we know about the world. A row is a program on a night, so that pair is never duplicated. You cannot occupy more beds than there are. The rate is what the two counts imply. And a bed holds one person while a room can hold several.

```
    # A row is a program on a night
    key = pl.struct(["occupancy_date", "program_id"])
    assert not df.select(key.is_duplicated()).to_series().any()

    # Relationships: what we know about the world
    assert (df["occupied"] <= df["capacity"]).all()
    implied_rate = 100 * df["occupied"] / df["capacity"]
    assert ((implied_rate - df["occupancy_rate"]).abs() < 0.01).all()
    # A room can hold more than one person, a bed cannot
    beds = df.filter(pl.col("capacity_unit") == "bed")
    assert (beds["service_users"] == beds["occupied"]).all()
    rooms = df.filter(pl.col("capacity_unit") == "room")
    assert (rooms["service_users"] >= rooms["occupied"]).all()
```

`scripts/00-simulate.py` simulates 100 programs over five nights and runs the tests. Writing the simulation is what forced the last test into existence, because to simulate the number of people in a room-based program we had to decide how people relate to rooms.

Clean

`scripts/01-clean.py` is then short. Names, then classes, then structure, then select.

```
cleaned = (
    raw
    .rename({c: c.lower() for c in raw.columns})
    .rename({"service_user_count": "service_users"})
    .with_columns(
        pl.col("occupancy_date").str.to_date("%Y-%m-%d"),
        pl.col("service_users").cast(pl.Int64),
        pl.col(
            "capacity_actual_bed", "capacity_actual_room",
            "occupied_beds", "occupied_rooms",
        ).cast(pl.Int64),
        pl.col("occupancy_rate_beds", "occupancy_rate_rooms").cast(pl.Float64),
    )
    .with_columns(
        pl.col("capacity_type")
        .replace_strict({"Bed Based Capacity": "bed", "Room Based Capacity": "room"})
        .alias("capacity_unit"),
        pl.coalesce("capacity_actual_bed", "capacity_actual_room").alias("capacity"),
        pl.coalesce("occupied_beds", "occupied_rooms").alias("occupied"),
        pl.coalesce("occupancy_rate_beds", "occupancy_rate_rooms")
        .alias("occupancy_rate"),
    )
    .select(list(SCHEMA))
)

# Count rows before and after
assert cleaned.height == rows_at_start

test_shelter(cleaned)
```

Two details. `cast` fails loudly if a value cannot be converted, which is what we want, because a stray "n/a" in a count should stop us. And `replace_strict` fails if it meets a capacity type that we did not list, where `replace` would pass it through silently.

```
cleaned <- read_parquet("data/cleaned_shelter.parquet")

glimpse(cleaned)
```

```
Rows: 500
Columns: 8
$ occupancy_date <date> 2026-09-16, 2026-09-16, 2026-09-16, 2026-09-16, 2026-
0~
```

```
$ program_id     <chr> "14911", "14891", "12291", "12292", "14671", "14651", "~
$ sector         <chr> "Youth", "Youth", "Youth", "Youth", "Youth", "Women", "~
$ capacity_unit  <chr> "bed", "bed", "bed", "bed", "bed", "bed", "bed", "bed",~
$ service_users  <int> 31, 8, 20, 32, 30, 43, 31, 31, 25, 40, 83, 12, 45, 32, ~
$ capacity       <int> 41, 8, 20, 33, 30, 43, 31, 31, 25, 40, 27, 12, 45, 35, ~
$ occupied       <int> 31, 8, 20, 32, 30, 43, 31, 31, 25, 40, 27, 12, 45, 32, ~
$ occupancy_rate <dbl> 75.61, 100.00, 100.00, 96.97, 100.00, 100.00, 100.00, 1~
```

All the tests pass on these 500 rows. That is five nights in September 2026. It tells us nothing about whether they would pass on 2021, when the data began and the system was under different pressures, and finding out is in the worksheet.

```
cleaned |>
  ggplot(aes(x = occupancy_rate)) +
  geom_histogram(binwidth = 2.5, boundary = 100) +
  facet_wrap(vars(capacity_unit), scales = "free_y") +
  labs(x = "Occupancy rate (per cent)", y = "Program-nights") +
  theme_minimal()
```

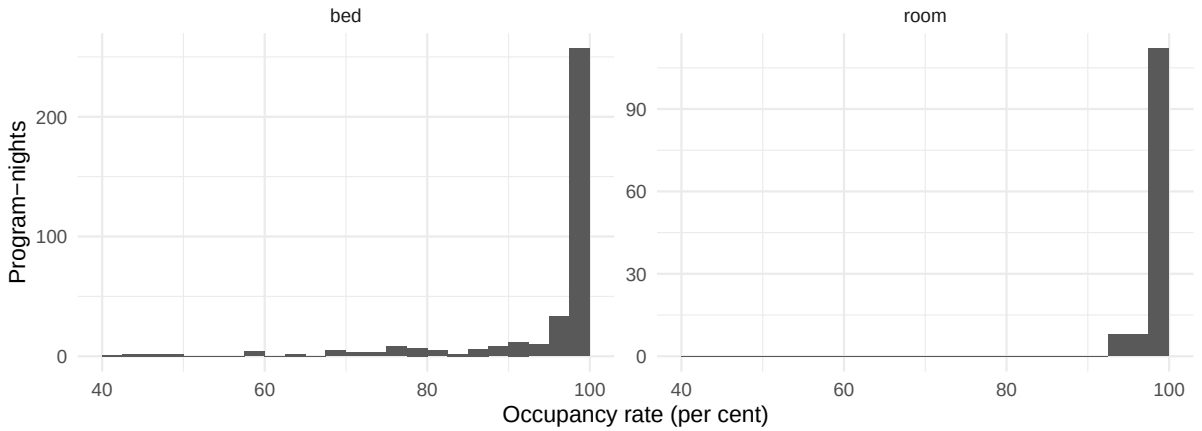


Figure 1: Occupancy rate of each program on each of five nights in September 2026, by whether capacity is counted in beds or rooms

Figure 1 is a check, of the kind that tells us what to think about next. Almost everything is full. For anyone who wants to model this later, that is a ceiling. The number of people in the system mostly tells us about the number of beds rather than the number of people who need one. It is a measurement point, from Week 5, that we found while cleaning.

Table 1: Data dictionary for the cleaned shelter dataset

variable	description	type	units	allowed_values
occupancy_date	Night to which the row refers.	date		2021-01-01 onwards
program_id	City of Toronto identifier for the shelter program. An identifier, not a quantity.	string		Five digits
sector	Group the program serves.	string		Families, Men, Mixed Adult, Women, Youth
capacity_unit	Whether the program's capacity is counted in beds or in rooms.	string		bed, room
service_users	Number of people staying in the program that night.	integer	people	0 or more
capacity	Number of beds or rooms (see capacity_unit) available that night.	integer	beds or rooms	1 or more
occupied	Number of beds or rooms occupied that night.	integer	beds or rooms	0 to capacity
occupancy_rate	occupied as a percentage of capacity.	float	per cent	0 to 100

Document

Data dictionary

The data dictionary is a CSV, `data/data_dictionary.csv`, that lives next to the data, so that it can be read by a person or by a machine.

```
readr::read_csv("data/data_dictionary.csv", show_col_types = FALSE) |>
  tt(width = c(2, 3.3, 0.8, 1, 1.9)) |>
  style_tt(fontsize = 0.8) |>
  format_tt(replace = "") |>
  format_tt(escape = TRUE)
```

The first page of a datasheet

Gebru et al. (2021) group their questions into motivation; composition; collection process; preprocessing, cleaning, and labeling; uses; distribution; and maintenance. Here are honest answers to a few, for this dataset. Notice how many of them are about the source rather than

about our code, and how many we cannot answer without reading the City's documentation, or asking.

For what purpose was the dataset created? The source data are published daily by the City of Toronto's Toronto Shelter and Support Services division, from its Shelter Management Information System, which is an administrative database. The City describes the data as unaudited, and as reflecting the state of each program's records, which may not be the actual situation in the program. Our cleaned version was created for teaching.

What do the instances represent? A shelter program on a night. It is not a person, and it is not a shelter, because one location can run several programs.

Is it a sample? Of programs, no, because it is meant to be every program that reports through the City's system. Of people experiencing homelessness, it is not a sample in any designed sense. It excludes people sleeping outside, people staying with others, and programs that do not use the City's system. It cannot be used to estimate the number of people who are homeless in Toronto.

Is any information missing? In the source, bed columns are empty for room-based programs and room columns for bed-based programs, by design. After cleaning, nothing is missing in the eight variables that we keep. We dropped 24 columns, including location.

Does the dataset relate to people? Could individuals be identified? It is counts rather than individuals. But some programs are small, and location is in the source, and a count of eight people in a named program at a known address on a known night is closer to identifying than it first appears. We did not keep location.

What preprocessing was done? Are the raw data saved? See `scripts/01-clean.py`. Yes, in `data/raw/`, as they arrived from the API, with the date of download in the file's history.

Is there anything a future user should know? Occupancy is nearly always close to capacity, and so the counts mostly reflect supply. And the City revises the data, so a download on another day may differ.

References

Alexander, Rohan. 2023. *Telling Stories with Data*. Chapman and Hall/CRC. https://tellingstorieswithdata.com.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, et al. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92. https://doi.org/10.1145/3458723.

Lewis, Crystal. 2024. *Data Management in Large-Scale Education Research*. 1st ed. Chapman Hall/CRC. https://datamgmtinedresearch.com/index.html.

Wickham, Hadley. 2014. "Tidy Data." *Journal of Statistical Software* 59 (10): 1–23. https://doi.org/10.18637/jss.v059.i10.